

KAFRIADA CORE

A solo build plan that holds the launch date and the pilot gate by cutting the feature list rather than the standard. Four stages, one deferred slice, and a critical path that starts with paperwork, not code.

TEAM	ARCHITECTURE	LAUNCH SCOPE	ESTIMATE	STATUS
1 — the CTO	Unchanged	Reduced, gate preserved	34–44 working days	For approval

01

The honest position

Solo changes the scope. It does not change the architecture.

Eight weeks was tight with three developers. Solo, the full pilot scope — eleven contexts, two tiers, roughly twenty-five tables, payments, a media pipeline, transfers, a feed, coordinator consoles, plus the operational work of a 27-LGA rollout — is a four-to-six month build. Saying otherwise would be telling you what you want to hear, and you would find out in week six anyway.

So the plan does the only honest thing available: **it holds the date and cuts the feature list**. Nothing about the standard moves. The money path is built to the same specification it would have had with three developers, because a ledger that is 80% correct is not 80% useful.

WHY THE ARCHITECTURE DOES NOT CHANGE

By the principle you set — business, then system, then architecture, then people — the system's properties have not changed because the team size did. The money boundary, the API as a product surface, and the separation of workloads are all still true of the system.

And there is a second reason, specific to working alone. **Solo, you have no code reviewer.** The three CI contract gates, the generated permission matrix and the database grants stop being process overhead and become the only thing standing between you and a silent mistake at 1am. The automated gates *are* your reviewer. That makes Stage 0's tooling investment more valuable when working alone, not less.

One scheduling fact worth stating plainly, because it is the thing most likely to break this plan: you have other active projects. A solo estimate assumes solo *attention*. If KAFRIADA gets three days a week rather than five, multiply every number below by 1.6 and plan the CEO conversation accordingly.

02 What launches, and what waits

The cut is chosen so that all three pilot gate metrics remain measurable.

The pilot exists to answer one question: **will athletes register for a free permanent ID, and will roughly 4% of them pay ₦2,500 to unlock a photo?** Everything that answers that question ships. Everything that does not, waits.

The constraint on cutting is the CEO's gate — 5,000 registrations, 4% paid conversion, 81 active clubs, verification inside 24 hours. Two of those four are about identity and money. The third is about clubs. So clubs cannot be cut entirely, and the plan keeps the minimum that makes "81 active clubs" a real number: club registration and rosters. **Transfers are the growth engine, not a gate metric** — they wait.

CAPABILITY	SLICE	REASONING
Registration, KUID, public profile, QR	LAUNCH	This is the product. Without it there is no pilot.
Phone OTP, sessions, RBAC with scope	LAUNCH	Identity anchor and the security floor. Scope must be in the first migration — it is one of the three expensive-to-change decisions.
Stage-2 payment, verification, review queue	LAUNCH	The conversion gate. This is the entire commercial hypothesis.
Assisted payment by coordinator	LAUNCH	Kept deliberately. Card penetration in Jigawa is low. Without a cash route the 4% conversion figure measures card ownership, not willingness to pay — which would make the pilot's headline number meaningless.
Club registration and rosters	LAUNCH	The "81 active clubs" gate. Minimum viable: register a club, invite an athlete, athlete accepts.
Reconciliation, expiry, integrity checks	LAUNCH	Not optional. These are what make the money trustworthy rather than merely functional.
Transfers	SLICE 2	Growth mechanism, not a gate metric. Nothing in the gate measures transfers, and the club-recruitment effect only matters once there are clubs to recruit from.
News feed with LGA targeting	SLICE 2	Engagement, not validation. Announcements can go out by SMS in the pilot.
Full coordinator console, settlement reporting	SLICE 2	A review queue plus a SQL-backed report covers 27 coordinators. Metabase against the read replica replaces a month of admin UI work.
Career history beyond registration	SLICE 2	The table ships in Slice 1 so no migration is needed later; only the UI waits.

THE CONVERSATION THIS CREATES WITH THE CEO

Not "we are late". The framing is: **same date, same gate, fewer features.** All four gate metrics remain measurable on the original launch date. Transfers and the feed move to a named Slice 2 that starts the week after Birnin Kudu goes live. If that trade is unacceptable, the alternative is honest and simple — fund the two other developers, or move the date. Those are the only three levers, and the choice is the CEO's to make with the numbers in front of them.

The critical path is paperwork

Three external approvals have multi-week lead times, none has been started, and all three block launch rather than code.

This is the single most schedule-critical finding in the plan, and it is the kind that is normally discovered in week six. **You can build for a month against test credentials. You cannot launch without these, and you cannot hurry them.** They start today, in parallel with Stage 0, and they are owned by whoever handles company administration — not by the person writing code.

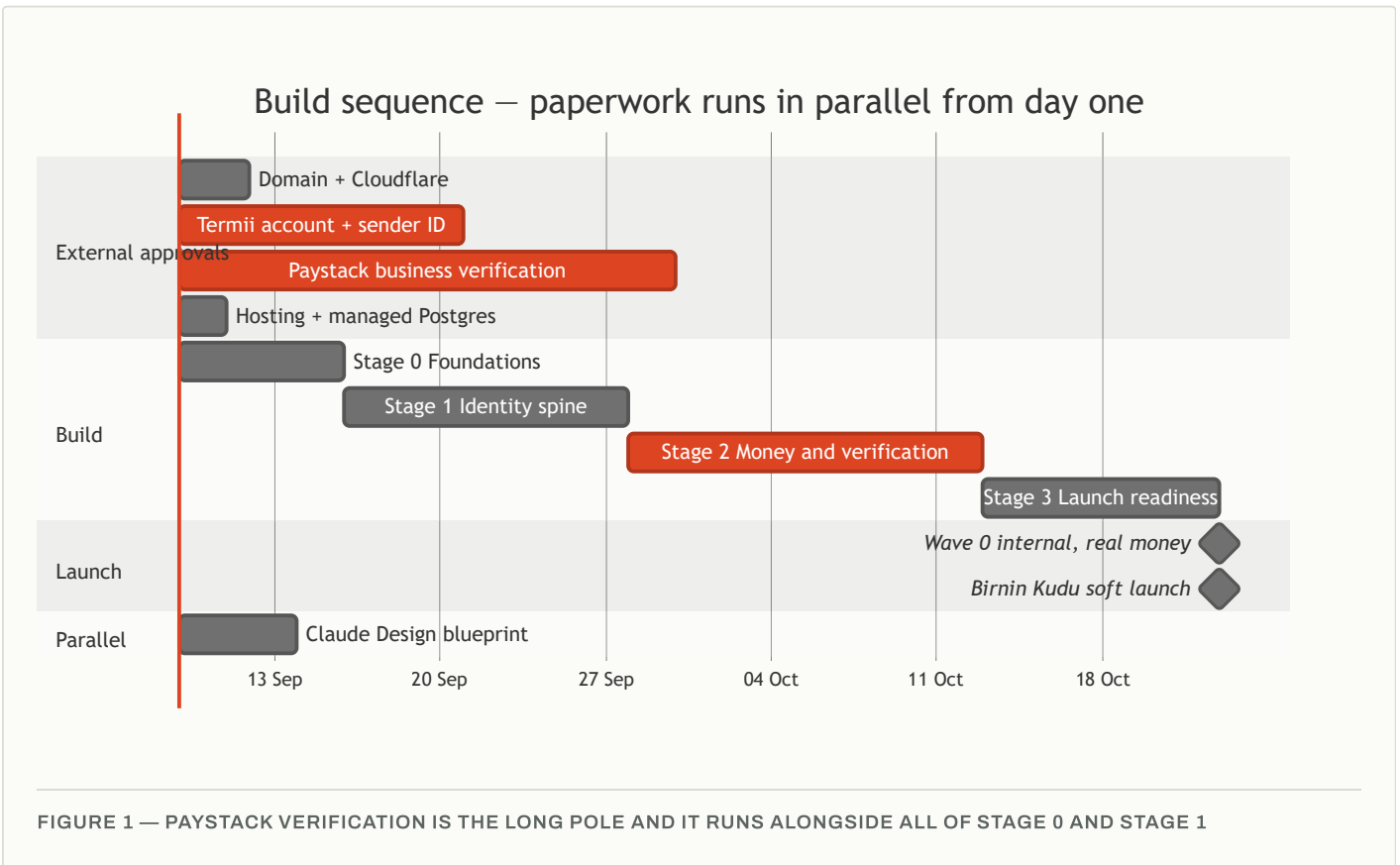
ITEM	LEAD TIME	BLOCKS	START TODAY BY
Paystack live account	1–3 weeks	All real money. Wave 0 cannot run without it.	Confirming KowaGuru's CAC registration is current, then submitting business verification with the corporate bank account. If CAC registration itself is incomplete, this becomes the longest item on the plan — find out this week, not in week five.
Termii sender ID	3–10 working days	OTP, which blocks registration, which blocks everything.	Creating the account and submitting the alphanumeric sender ID for approval. Register a fallback numeric route so a rejected sender ID does not stop the pilot.
Domain + Cloudflare	1–3 days	Staging deploy, edge caching, TLS posture.	Registering the domain and adding the Cloudflare zone. Do this first of the three — it is the fastest and the other work wants a real hostname.
Hosting + database	Same day	Staging environment.	Render or Railway account, Frankfurt region, plus managed Postgres with point-in-time recovery. Card on file.
NDPR filing owner	—	Nothing technical. Legal exposure only.	Naming a person. Still assumption A7 with no assignee — it crosses the 2,000-subject threshold at 5,000 athletes.

DO THIS BEFORE YOU WRITE A LINE OF CODE

Register the domain, open the Termii account and submit the sender ID, and start Paystack business verification. Total effort: perhaps two hours. Total schedule risk if deferred to week four: **the launch date.**

04 Stage 0 — Foundations

Nothing user-visible ships. Everything after this depends on it being right.



- ☐ **Decide where KAFRIADA lives.** Right now `C:\Users\HP` is itself a git repository whose history is Lucid HR work, and KAFRIADA sits inside it untracked. Building here puts one client's source into another client's repository.
- ☐ **Recommended:** initialise a fresh repository at `C:\Users\HP\KAFRIADA`, and add `KAFRIADA/` to the parent repository's `.gitignore` so the two can never entangle. Push to a private remote the same day — a solo build with no remote is one disk failure from nothing.
- ☐ **Move the six PDFs and the prompts folder** into `docs/` inside the new repository so the specification travels with the code.
- ☐ **Branch protection on `main`** even working alone: no direct pushes, CI must pass. Solo, this is what stops a tired 1am commit going straight to production.

0.2 — Local environment

HALF A DAY

- ☐ **Install Docker Desktop.** Not currently on this machine. Needed for Postgres 16, Redis, MinIO and a mail catcher locally.
- ☐ **Install Python 3.12 and uv.** Current PATH Python is 3.11.4; `py` has 3.13.7. Pin 3.12 to match production exactly. `uv` for dependency management — one fast tool, lockfile included, and it removes the virtualenv fiddling that eats solo time.
- ☐ **Node 22.18 is already present** and correct for Next.js 15. Add `pnpm`.
- ☐ **`docker-compose.yml`** — postgres:16, redis:7, minio, mailpit. One command from clone to running.

0.3 — Repository skeleton

1 DAY

- ☐ **Monorepo layout:** `api/` (FastAPI, and the worker's entypoint), `web/` (Next.js), `infra/` (compose, deploy config), `docs/`, `contracts/` (the generated OpenAPI document, committed).
- ☐ **FastAPI skeleton** with settings from environment, a health endpoint, structured JSON logging, and request-ID middleware that returns the ID in a response header.
- ☐ **Context package layout** under `api/contexts/` — one package per bounded context, each exposing exactly one `service.py`. Empty but correctly shaped, so the boundary exists before there is anything to violate it.
- ☐ **Next.js skeleton** with the server-side API client, session cookie handling, and a deliberately plain first page. No component library yet.

☐ **Alembic** , with the migration test harness that runs every migration initialised up and down against a scratch database in CI.

☐ **Three schemas:** `identity` , `money` , `ops` .

☐ **Four roles with grants:** `kaf_app` , `kaf_money` , `kaf_reader` , `kaf_migrate` . This is where the isolation guarantee is actually created.

☐ `ops.audit_log` — no `UPDATE` or `DELETE` on `Plus` , `kaf_app` and `ops.audit_log` with `INSERT` role in it. `the` attempting to asserting `and SELECT` the `test` a delete as the failure. `only` system `that` Without `holds` `proves` it `it is a claim, not a control.`

☐ `ops.users` and `ops.user_roles` One of the three most expensive carrying `scope_type` and `scope_id` things to change later. It costs from this first migration. two columns today.

☐ **A money type helper** that stores `BIGINT` kobo as and refuses anything that is not an integer, so no float can ever reach the database.

0.5 — The three CI gates

1–2 DAYS

- ☐ **Gate 1 — contract freshness.** Regenerate OpenAPI from the `contracts/openapi.json` FastAPI app and fail if it differs from the committed
- ☐ **Gate 2 — breaking-change detection.** Diff the new contract against the previous one; fail on a removal or type change unless the commit carries an explicit version-bump marker.
- ☐ **Gate 3 — client compilation.** Generate the TypeScript client and typecheck `web/` against it, so a domain change that breaks the UI fails in the same commit.
- ☐ **Plus the standing checks:** `ruff` and `api/`, `ESLint` and `web/`, `pytest`, the `service.py` migration up-down test, and an import-linter contract asserting no context imports past another's
- ☐ **And the route manifest test:** every route must declare a required permission, or the build fails. It asserts nothing today because there are no routes — which is exactly when to add it.

0.6 — Deploy pipeline

1 DAY

- ☐ **Staging in Frankfurt:** three services — web, api, worker — plus managed Postgres with point-in-time recovery, from the first day.
- ☐ **Migrations as a gated release step** under `kaf_migrate`, running before the API takes traffic.
- ☐ **Deploy order enforced:** migrations, then api, then web. Wrong order is the classic two-tier outage.
- ☐ **Sentry and OpenTelemetry wired** before there is anything to trace. Retrofitting tracing is miserable; adding it to an empty app takes an hour.
- ☐ **Secrets in the host's secret store**, `gitleaks` as a pre-commit hook and in CI.

STAGE 0 EXIT CRITERIA

A commit to `main` runs all gates, deploys three services to staging in Frankfurt, applies migrations under the migration role, and serves a health endpoint over your own domain through Cloudflare. A test asserts that `kaf_app` cannot delete an audit row. **You should be able to destroy your laptop and be productive again in an hour.**

05 Stage 1 — The identity spine

Ends with a real person holding a real printed QR card. Roughly 12 working days.

1.1 — access, geography, audit

3–4 DAYS

- ☐ **Users, credentials, sessions.** `argon2id` **outside** any transaction, in a sized thread pool with a concurrency semaphore. Sessions in Postgres — 30-minute idle for staff roles, 30 days for athletes.
- ☐ **Roles, permissions, scope.** Table-driven, seeded with the 8 live roles. `rbac.can(user, permission, scope)` as a FastAPI dependency, deny by default.
- ☐ **The generated permission matrix test.** The role-by-route fixture from the architecture document, checked into the repo, with cross-tenant cases — club_admin of A attempting B, coordinator of Birnin Kudu attempting a Dutse athlete. It grows with every route from here on.
- ☐ **Geography seeded** with Jigawa and all 27 LGAs, each with an `is_live` flag, all false except Birnin Kudu.
- ☐ `audit.write()` in use from the first business action, not added later.

1.2 — identity and the KUID

3–4 DAYS

- ☐ **Athletes, `kuid_counters`, `career_events`.** Unique constraint on the phone number — the identity anchor.
- ☐ **The mint:** the single atomic `INSERT ... ON CONFLICT ... RETURNING`, positioned as the second-to-last statement in the transaction, with hashing already done outside it.
- ☐ **Retry safety:** a duplicate phone raises `23505`, which is caught specifically and returns the existing KUID rather than an error.
- ☐ **The burst test:** 200 concurrent registrations including deliberate duplicates, asserting `COUNT(*) = COUNT(DISTINCT kuid)` and no gaps. **Write this test in Stage 1, not Stage 3** — it is the proof the whole register depends on.

- ☐ **Registration as a server-rendered form POST.** No JavaScript required — test it with JavaScript disabled before calling it done, because Opera Mini will.
- ☐ **OTP through Termii** via the notifications outbox, so a Termii outage queues rather than fails. Rate limited 1 per 60s and 5 per day per phone.
- ☐ **Login, logout, password reset.**
- ☐ **Public profile at `/a/{kuid}`** — anonymous, server-rendered, edge-cached 60s with the query string in the cache key. Silhouette unless approved.
- ☐ **QR generation** with the versioned signature `?s=1. ...`, plus a bulk print sheet for coordinators. The page wording says "Issued by KAFRIADA — check the photo matches the bearer", not "verified athlete".

STAGE 1 EXIT CRITERIA

You register yourself on a real phone over a real network, receive a real OTP, and print a QR card that a stranger can scan and read on a browser with JavaScript turned off. The burst test passes. Every action so far has an audit row.

06

Stage 2 — Money and verification

The part that cannot be fixed later. Roughly 15 working days, and the one place not to compress.

2.1 — ledger and payments

4–5 DAYS

- ☐ **Wallets and insert-only ledger_entries** in integer kobo, written `kaf_money` role. only by the
- ☐ **Payment intents** with our own reference `KAF-{uuid}`, and Paystack initialise against test keys.
- ☐ **The webhook:** raw body read before any parsing, HMAC-SHA512 verified in constant time, then the idempotency insert `ON CONFLICT DO NOTHING RETURNING` — no row returned means seen before, return 200 and do nothing.
- ☐ **The amount check.** Currency NGN, status success, and amount equal to the expected kobo. A mismatch freezes and alerts; it never approves.
- ☐ **Gross and provider fee as two separate ledger lines** from the very first payment.
- ☐ **Replay test:** send the same webhook five times concurrently and assert exactly two ledger rows exist.

2.2 — media and verification

4–5 DAYS

- ☐ **Private R2 bucket.** Presigned PUT for upload; the asset row is written only after the app confirms the object exists.
- ☐ **Worker derivative pipeline:** re-encode, strip EXIF, and serve only the derivative through a permission-checked signed read.
- ☐ **Verification cases** with the full state machine including `revoked` and `escalated`, and the resubmission counter capped at three.
- ☐ **The reviewer rule in the service, not the UI:** no approving your own record or an athlete of a club you administer.
- ☐ **A deliberately plain review queue** — a list, a document viewer, approve, reject with reason. Four decisions a day does not need a designed console.

2.3 — outbox, workers and the safety net

3–4 DAYS

- ☐ **The outbox table and consumer** using `FOR UPDATE SKIP LOCKED`. All notifications move onto it.
- ☐ **Hourly reconciliation** against Paystack's verify API, sharing the exact code path the webhook uses. It may only ever confirm a credit — never reverse, refund or cancel. Amount mismatches freeze for a human.
- ☐ **Daily expiry** — 72 hours, and only for records with no successful charge.
- ☐ **Nightly integrity check:** every wallet balances, every athlete has exactly one KUID, no KUID appears twice, every media row points at an object that exists. Pages on failure.
- ☐ `wallet.record_reversal()` — super_admin, re-auth, mandatory reason, records a refund made by hand in Paystack. No endpoint moves money.

☐ **Coordinator pays on behalf:** ledger lands on the athlete, `coordinator_id` tagged on the payment, daily caps on count and naira, and an SMS receipt to the athlete's phone at confirmation.

☐ **Club registration, roster invite, athlete accepts.** The at-most-one-open-membership invariant as a partial unique index — the database enforces it, not the code.

☐ **Settlement figures via Metabase** against the read replica. SQL, not a built screen.

STAGE 2 EXIT CRITERIA — DO NOT SOFTEN ANY OF THESE

A real ₦2,500 moves through Paystack live and lands correctly in the ledger as two lines. The same webhook replayed five times changes nothing. A deliberately wrong amount is rejected and alerts. The nightly integrity check is green. A coordinator pays for someone else and that person's phone receives the receipt. **Stage 3 does not begin until every one of these is true.**

Stage 3 — Launch readiness

The work that makes it safe to put real people on it. Roughly 10 working days.

COMPLIANCE

Privacy notice with recorded consent — a row carrying the notice version and timestamp, not just a displayed link. Wording that says plainly that deletion means anonymisation because audit and financial records must be retained. Export and anonymise as scripts with audit rows.

SECURITY SWEEP

Full permission matrix green across all roles and routes including cross-tenant cases. Rate limits verified. TLS Full-strict with authenticated origin pulls so the origin refuses anything not from Cloudflare. Secret rotation procedure written and tested once.

RESILIENCE

The eight-point restore drill, including that the ledger balances in the restored database, that R2 and the database still agree, and that the wall-clock time is written down — **that number is your RTO**. Dependency-failure drills: kill Paystack, Termii and R2 in staging and confirm the user sees the right message.

OBSERVABILITY

Sentry live. The three dashboards built — Money, Funnel, Health — with Metabase on the first two. The alert list split into page-someone and open-a-ticket. Solo, alerting is not a nicety: it is the only thing watching while you sleep.

PERFORMANCE

Load test at 500 virtual users with the mixed profile. Lighthouse in CI with the 150KB first-load JavaScript budget enforced. Every critical flow retested with JavaScript disabled.

OPERATIONS

Coordinator runbook and training material. Incident runbook. Reconciliation-mismatch runbook. A printed receipt book and a posted complaint number for every LGA — the assisted-payment controls software cannot provide.

THEN LAUNCH, IN THE SEQUENCE ALREADY AGREED

Wave 0 — team plus 20 invited athletes, three days, real money in live mode, including one deliberate refund and one deliberately duplicated webhook. The restore drill runs here, before any member of the public is exposed. **Wave 1** — Birnin Kudu only, two weeks, with you physically present at a registration drive watching people use it. Waves 2 to 4 open behind the `is_live` flag, gated on the five numeric criteria, no deploy required to open or close one.

Slice 2 — after Birnin Kudu is live

Named and scheduled, so it reads as sequenced rather than cancelled.

Starts the week after Wave 1 goes live, while the pilot runs and produces the numbers the CEO gate needs. Roughly four to six weeks solo, and the natural point to bring in the other developers if the pilot's early figures justify the spend.

- **Transfers** — the full state machine including `awaiting_buyer_onboarding`, and the completion transaction with all five effects atomic. The club-recruitment gate is the growth engine, and it wants clubs already on the platform to work at all.
- **News feed** with state and LGA targeting, and the automatic transfer announcement through the outbox.
- **Coordinator console** proper — settlement reporting, per-LGA statistics, bulk QR printing improvements.
- **Career history UI** on the public profile. The table already exists from Slice 1, so this is presentation only.
- **Data export automation** if NDPR requests exceed roughly one a week.

09 Risks, and what I would cut next

Ordered by likelihood of actually happening.

RISK	LIKELIHOOD	RESPONSE
Attention is split across other projects	HIGH	The single most likely cause of slip. Decide now how many days a week KAFRIADA genuinely gets, multiply the estimates, and tell the CEO that number rather than the optimistic one.
Paystack verification runs long	MEDIUM	Started day one. Everything up to Wave 0 can be built on test keys, so it only bites at launch — but then it bites completely. Escalate at day 10 if there is no movement.
Termii sender ID rejected	MEDIUM	Register a numeric fallback route at the same time. OTP over a generic number is worse branding and entirely survivable.
Stage 2 overruns	MEDIUM	Do not compress Stage 2. Take the time from Stage 3's polish or from Slice 2 — never from the money path. A ledger built in a hurry is the one defect this product cannot absorb.
Solo, no code review	MEDIUM	The CI gates are the mitigation and that is why Stage 0 is not skippable. Add a weekly self-review against the architecture document's correction list.
Coordinator cash handling goes wrong in Wave 1	MEDIUM	Anticipated by design: caps, athlete-facing receipts, settlement figures, and the state coordinator calling five random athletes per LGA per week.
The blueprint contradicts the build	LOW	It arrives during Stage 0 and Stage 1, when almost nothing is built. Reconcile it at the Stage 1 boundary; the architecture PDFs are the tiebreaker.

IF IT SLIPS, CUT IN THIS ORDER

1. Slice 2 slips further — it is already after launch. 2. Coordinator tooling becomes pure Metabase SQL. 3. Club registration becomes an admin-entered record rather than self-service, which costs the CEO's "81 clubs" nothing since coordinators can enter them. 4. Waves 3 and 4 slip, so the pilot runs in fewer LGAs for longer.

Never cut: the reconciliation job, the nightly integrity check, the permission matrix test, the restore drill, or the burst test. Those five are what make the difference between a pilot and a liability.

To start

Three decisions from you, then I begin Stage 0.

1. **Approve the scope cut** — transfers and the feed move to Slice 2, everything else launches. Or tell me to keep full scope and move the date instead.
2. **Confirm the repository decision** — a fresh git repository at `C:\Users\HP\KAFRIADA`, with `KAFRIADA/` added to the parent repository's ignore file, pushed to a private remote today.
3. **Start the three external approvals** — domain, Termii sender ID, Paystack business verification. Two hours of paperwork that determines whether the launch date is real.

On approval I begin with 0.1 and 0.2 — repository, ignore rules, remote, Docker Compose, toolchain — and then migration 001, which is where the schema separation, the four database roles, the audit grants and the RBAC scope columns get created. **Those are three of the decisions that are most expensive to change later, and they all land in the first migration.**

THE SHAPE OF THIS PLAN IN ONE PARAGRAPH

Paperwork starts today because it is the longest pole and nobody notices it until week six. Stage 0 buys the automated gates that replace the code reviewer you do not have. Stage 1 ends when a real person holds a printed card. Stage 2 is the money and it does not get compressed. Stage 3 makes it safe for strangers. Transfers and the feed are named, scheduled and honest about being second — because the pilot's question is whether people will register and pay, and every day spent on anything else is a day not spent answering it.